

Weeks 3–4: Design and Modeling

Objective of these two weeks:

Produce all design deliverables before writing production code:

class diagrams, LangGraph workflows, diagrams.

These deliverables constitute the “architectural memory” of the project.

Week 3 (from 16/02/2026 to 20/02/2026) Class Diagram and Data Modeling:

Note: The class diagram is still being worked on based on needs and the database migrations are being saved.

During the third week of my internship, I focused on designing the data architecture of the project by creating the complete class diagram and modeling the database structure. I designed a comprehensive class diagram composed of Django models organized across seven application layers: Authentication, Core, Specification/RAG, Test Management, Automation, Integrations, and AI. I defined the hierarchical structure of the platform as Organization → Team → Project → Specification → Suite → Scenario → TestCase, which reflects how testing artifacts are organized within the system. I also modeled the relationships between entities using concepts such as ForeignKey, OneToOne, and ManyToMany relationships and analyzed their implications for database design and performance. Several key models were added to support the AI system, including AIAgentRun, AIAgentStep, AgentMemory, and AgentEvalScore. During this process, I made the architectural decision to keep the Organization model even though the platform currently has only one client (BIAT), in order to ensure future extensibility. This phase allowed me to learn important design principles in Django, such as the rule that a “many” relationship should not be stored on the “one” side since the foreign key already represents the relationship. I also understood that the VectorField used in the SpecChunk model through pgvector is the central component of the RAG pipeline, and that the AgentMemory model allows the system to learn from human corrections without requiring fine-tuning. Additionally, the separation between Organization, Team, and UserProfile enables a clean multi-tenant architecture, while nullable fields such as ai_api_key or jira_token make it possible to run the platform without AI or external integrations. One of the main difficulties I faced was balancing a complete architecture with the risk of over-engineering for an academic project. To address this, I designed models that meet BIAT’s current needs while remaining extensible without requiring heavy database migrations in the future. Designing coherent models without circular dependencies taught me to think in logical architectural layers before thinking in SQL tables. I realized that each modeling decision impacts the entire system, and that spending one hour thinking carefully about the schema can prevent a week of broken migrations later.

Week 4 (from 23/02/2026 to 27/02/2026) LangGraph Workflows and Diagrams

During the fourth week of my internship, I focused on designing the AI agent workflows and producing the necessary diagrams to clearly describe the system behavior. I designed three LangGraph workflows corresponding to the main agents of the platform: the Test Generation Agent, the Browser Execution Agent, and the Code Review Agent. For each agent, I defined the

state schemas using TypedDict and identified the Interrupt Nodes that allow Human-in-the-Loop interactions, enabling the system to pause and wait for human validation when necessary. I also created a sequence diagram describing the two main system flows: the first for automated test generation and the second for live browser execution. In addition, I produced a use case diagram that includes five actors and six main functional areas of the platform. Through this work, I learned that LangGraph operates as a directed graph with persistent state, where each node transforms the state of the system. I also understood how Interrupt Nodes enable human supervision by suspending the workflow until an external action is performed. I discovered that the Browser Execution Agent follows a cyclic observe–decide–act loop rather than a linear process, and that Human-in-the-Loop communication is implemented using Django Channels, which sends pause events to the React frontend through WebSockets.

One difficulty I encountered was representing the cyclic behavior of the execution agent within a linear sequence diagram. To solve this, I modeled the observe–decide–act loop as an iterative process with a return arrow indicating repetition. Creating these diagrams forced me to think carefully about every interaction between system components. I discovered that the Browser Execution Agent is the most complex part of the platform, not primarily because of the AI logic, but because of the real-time WebSocket streaming and the pause and resume mechanisms required for human supervision not to mention the additional Ai costs (tokens per execution).